

CineSeat Backend — İş Planı

3 Kişi / CQRS Design Pattern · Tek Proje Birleştirme · Visual Studio Geçiş

Bu doküman CineSeat backend'inin CQRS design pattern ile geliştirilmesi için hazırlanan iş planını, mevcut çok-projeli yapının tek projeye birleştirilmesini ve Visual Studio'ya geçiş adımlarını içerir.

Mevcut Durum

Backend şu an **4 ayrı proje + 1 solution** yapısında:

```
backend/
├─ CineSeat.slnx                → 4 projeyi bağlayan solution
├─ CineSeat.Domain/             (bağımsız) → Entities, Enums, BaseEntity
├─ CineSeat.Application/        (→Domain) → ŞU AN BOŞ (sadece csproj)
├─ CineSeat.Infrastructure/      (→App,Domain) → ApplicationDbContext, Migrations, EFCore+Npgsql
├─ CineSeat.Api/                 (→App,Infra) → Program.cs, Controllers, appsettings
└─ dotnet-tools.json             → dotnet-ef 10.0.10
```

Bağımlılık zinciri: **Api** → **Application** + **Infrastructure** → **Domain**. Migration'lar Infrastructure'da.

BÖLÜM A — 4 Proje → Tek Proje Birleştirme

Karar: Tek Proje vs 4 Proje

4 projeyi tek projeye indirmek **teknik olarak sorunsuz** ve küçük ekipler için çok yaygın. Tek kaybedilen şey derleyicinin katman kuralını zorlaması:

	4 proje (şimdiki)	Tek proje (hedef)
Onion katmanları	Derleyici zorlar — Domain, Infrastructure'a referans veremez	Katmanlar klasör olur — kural disiplin/gelenek, derleyici engellemez
Karmaşıklık	4 csproj, referans yönetimi	1 csproj, çok daha sade
Küçük ekip için	Gereğinden ağır olabilir	✅ Daha pratik
EF migration komutları	--project / --startup-project gerekli	Bayrak gerekmez, sadeleşir

Sunum açısından hâlâ **"Onion/katmanlı mimari"** denebilir — sadece "katmanları ayrı proje yerine ayrı klasörle uyguladık" denir. Prensip (bağımlılık içeri akar) aynı kalır, ekip disiplinle korur.

Ara yol (opsiyonel): En sade profesyonel orta nokta 2 projedir → **CineSeat.Api** (web) + **CineSeat.Core** (kalan her şey). Bu plan tek proje hedefine göre kurulmuştur.

Birleştirme Sonrası Yapı (namespace'ler AYNI kalır)

C# namespace'i projeden bağımsızdır. `namespace CineSeat.Domain.Entities` gibi isimler **korunabilir**, sadece dosyalar tek projeye taşınır. Böylece **migration'lar ve Program.cs hiç bozulmadan** çalışmaya devam eder.

```
backend/
├─ CineSeat/                                ← TEK proje (Microsoft.NET.Sdk.Web)
│   ├── CineSeat.csproj                    ← tüm NuGet paketleri burada toplanır
│   ├── Program.cs
│   ├── appsettings.json
│   └─ Domain/
│       ├── Entities/ (+ Common/BaseEntity.cs)
│       ├── Enums/
│       ├── Application/                  ← CQRS burada yaşayacak (Bölüm C)
│       │   ├── Common/
│       │   └─ Features/
│       └─ Infrastructure/
│           ├── Data/ApplicationDbContext.cs
│           └─ Migrations/                ← mevcut 3 migration korunur
└─ Api/
    └─ Controllers/
```

Birleştirme Adımları

1. `backend/CineSeat/` adında tek yeni proje oluştur (`dotnet new webapi`), veya `CineSeat.Api` 'yi ana gövde yapıp diğerlerini içine taşı.
2. `Domain/` , `Application/` , `Infrastructure/` klasörlerini (Entities, Enums, Data, Migrations) bu projenin içine taşı — **namespace'lere dokunma**.
3. Tüm NuGet paketlerini tek `.csproj` 'de topla: `Npgsql.EntityFrameworkCore.PostgreSQL` 10.0.0, `EFCore.NamingConventions` 10.0.1, `Microsoft.EntityFrameworkCore.Design` + `.Tools` 10.0.7, `Microsoft.AspNetCore.OpenApi` 10.0.7, (+ MediatR, FluentValidation).
4. Örnek `WeatherForecast.cs` / `WeatherForecastController.cs` dosyalarını **sil** (temizlik).
5. `dotnet-tools.json` 'ı koru (dotnet-ef aynen çalışır).
6. Eski 3 `.csproj` + `CineSeat.slnx` 'i sil, yeni tek proje için temiz bir `.sln` üret (`dotnet new sln` + `dotnet sln add`).
7. **Doğrulama (şart):** `dotnet build` yeşil mi + `dotnet ef migrations list` mevcut 3 migration'ı görüyor mu + `dotnet ef database update` sorunsuz mu.

EF için ne değişir: Artık migrations assembly = tek proje olduğu için komutlar sadeleşir:

```
dotnet ef migrations add AddCqrsFeatures    # --project/--startup-project'e gerek yok
dotnet ef database update
```

BÖLÜM B — VS Code'dan Visual Studio'ya Geçiş

Önemli: **"geçiş" dosya taşımak değil.** Visual Studio ve VS Code aynı `.csproj` / `.sln` dosyalarını kullanır. Projeyi kopyalamaya gerek yok — sadece Visual Studio'da açarsın.

1. .NET 10 Uyumu (dikkat!)

Proje `net10.0` hedefliyor. **.NET 10 en yeni Visual Studio'yu gerektirir** — eski **Visual Studio 2022, .NET 9'a kadar** destekler.

- **Visual Studio 2026 Community** (ücretsiz) kur. Kurulumda **"ASP.NET and web development"** iş yükünü (workload) seç.
- Doğrulama: proje açıldığında "unsupported target framework net10.0" uyarısı çıkmamalı. Çıkarsa VS sürümü .NET 10'u desteklemiyordur → güncelle.

2. Projeyi Açma

- Birleştirmeden **sonra**: `File → Open → Project/Solution` → yeni tek `CineSeat.sln` 'i (veya doğrudan `CineSeat` klasörünü) aç.
- Birleştirmeden **önce**: `.slnx` yeni bir formattır; eski VS'te açılmazsa `dotnet sln` ile klasik `.sln` üret/dönüştür. (Tek projeye geçince bu dert kalmaz.)

3. Çalıştırma Ayarı

- **Startup project**'i (tek proje / Api projesi) sağ tık → *Set as Startup Project*.
- Yeşil ▶ (veya F5) ile çalışır; Swagger/OpenAPI otomatik açılır.
- `appsettings.json` 'daki PostgreSQL bağlantı dizesi zaten hazır — pgAdmin/postgres'in açık olması yeterli.

4. Migration'lar — VS'te İki Yol

Yol	Komut	Nerede
Package Manager Console (VS'e özel)	<code>Add-Migration Ad</code> , <code>Update-Database</code>	Tools → NuGet Package Manager → Package Manager Console
Terminal (alışkın olduğun)	<code>dotnet ef migrations add Ad</code> , <code>dotnet ef database update</code>	View → Terminal

İkisi de aynı işi yapar. "Default project" açılır menüsünden doğru projeyi seçmek yeterli.

5. Git

`.gitignore` 'da `bin/` , `obj/` , `.vs/` , `*.user` zaten var — VS'in ürettiği çöp dosyalar commit'e girmez. Ek ayar gerekmez.

BÖLÜM C — 3 Kişilik CQRS İş Planı

CQRS Bizde Ne Demek?

Her işlemi **Command** (yazma: create/update/delete) ve **Query** (okuma) olarak ayırırız. Her biri kendi **Handler**'ına sahiptir. Onion'ın **Application** katmanına oturur:

```
Application/  
├─ Common/           (Result<T>, Behaviors, IApplicationDbContext arayüzü)  
└─ Features/  
    └─ Movies/  
        ├─ Commands/CreateMovie/ → CreateMovieCommand.cs + Handler + Validator  
        └─ Queries/GetMovies/    → GetMoviesQuery.cs + Handler + MovieDto
```

Araç Seçimi

Seçenek	Artı	Eksi
MediatR + FluentValidation standart	Hazır, çok kaynak var; assembly-scan sayesinde yeni handler eklerken ortak dosyaya dokunulmaz (merge conflict az)	MediatR son dönemde ticari lisansa geçti — öğrenci/küçük proje için ücretsiz tier olabilir, teyit edilmeli
Manuel CQRS (kendi ICommand / IQuery + handler arayüzü)	Sıfır bağımlılık, öğrenmek için ideal	Biraz daha fazla el emeği (DI kaydı, dispatcher)

Öneri: Öğrenme + sunum bağlamında MediatR ile gitmek standart cevaptır; lisans hızlıca kontrol edilmeli. Takılınca manuel CQRS'e kısa sürede dönülebilir.

Faz 0 — Ortak Altyapı birlikte · lead: Berke · ~1-2 gün

Bunlar bittikten sonra paralel çalışma başlar:

- Tek projeye birleştirme + Visual Studio kurulumu (Bölüm A + B)
- MediatR + FluentValidation kur, DI'a kaydet (**AddMediatR(assembly)** , **AddValidatorsFromAssembly**)
- Ortak parçalar: **Result<T>** / **ApiResponse** , global exception middleware, **ValidationBehavior** (pipeline), **BaseApiController** , **IApplicationDbContext** arayüzü
- Migration'ın tek projede çalıştığını doğrula

Faz 1 — Dikey Dilim Şablonu birlikte · ~0.5-1 gün

- Bir feature'ı **uçtan uca** yap (ör. Movies: **CreateMovieCommand** + **GetMoviesQuery** + **MoviesController**). Herkes bunu kopyalayıp kendi modülüne uyarlar. **Bu şablon standardı kurar** — 3 kişi farklı stil yazmaz.

Faz 2 — Paralel Geliştirme 3 kişi · ayrı branch · ~1-1.5 hafta

Modül bazlı bölüşüm (CQRS feature'a göre ayrılır, çalışma minimum):

Kişi	Modül	Entity'ler	Ana Command/Query'ler
A	Kimlik & Sosyal	User, Role, Permission, RolePermission, UserFavorite, Comment	Register, Login, AssignRole, AddFavorite, AddComment; GetProfile, GetUserFavorites, GetComments
B	Katalog & Konum	Movie, Genre, MovieGenre, Campaign, Cinema, City, District	CreateMovie, UpdateMovie, CreateCampaign, CreateCinema; GetMovies(+filtre), GetMovieDetail, GetCinemasByCity, GetCampaigns
C ⚠	Salon & Rezervasyon	Hall, Technology, HallTech, Seat, Showtime, SeatLock, Reservation, Ticket	CreateShowtime, LockSeat, CreateReservation, IssueTicket; GetShowtimes, GetSeatMap, GetMyReservations

⚠ **C en zor modül** — koltuk kilitleme (SeatLock) eşzamanlılık gerektirir. İyi haber: `ShowtimeId + SeatId` üzerine eklenen **unique index** çift rezervasyonu DB seviyesinde zaten engelliyor; C bunun üstüne kod mantığını kurar. Yükü dengelemek için B'den birkaç basit CRUD C'ye kaydırılabilir, JWT auth A'ya verilebilir.

Faz 3 — Entegrasyon & Kalite birlikte · ~3-4 gün

- JWT authentication + RBAC authorization (Permission tabloları hazır)
- Seed data (roller, izinler, türler — `HasData`)
- Swagger düzenleme, örnek istekler
- Temel testler + uçtan uca deneme
- README / kısa API dokümanı

⚠ Tek Projede 3 Kişi → Merge Conflict Tuzağı

En büyük risk `Program.cs` ve `.csproj` 'de çakışma. Önlem:

- **MediatR assembly-scan** kullanın → yeni handler eklerken `Program.cs` 'e dokunulmaz (otomatik bulunur). Bu tek başına çakışmaların çoğunu bitirir.
- Her kişi **kendi klasöründe** çalışır (`Features/Movies/` , `Features/Users/` ...), controller'lar ayrı dosyalar → aynı satıra iki kişi dokunmaz.
- DI kaydı gerekirse: her katman/modül kendi `DependencyInjection.cs` uzantı metodunu yazsın, `Program.cs` sadece onları çağırırsın.

Git Akışı (3 Kişi)

```
main (korumalı)
├─ feature/identity      (A)
├─ feature/catalog      (B)
└─ feature/booking      (C)
```

- Faz 0 + Faz 1 main'e girmeden kimse dallanmaz (herkes aynı şablonu alsın).
- Her kişi kendi branch'inde çalışır → PR → en az 1 kişi review → main'e merge.
- Sık sık `main` 'i kendi branch'ine çek (conflict'i küçük tut).

Kaba Zaman Çizelgesi

Faz	Süre	Kim
0 — Altyapı	1-2 gün	Berke (lead) + ekip
1 — Şablon	0.5-1 gün	Birlikte
2 — Paralel	1-1.5 hafta	A / B / C ayrı
3 — Entegrasyon	3-4 gün	Birlikte
Toplam	~2-3 hafta	(tempoya göre)

Definition of Done (her feature için)

Command/Query + Handler ✓ · Validator ✓ · Controller endpoint ✓ · DTO (entity'yi ham döndürme) ✓ ·
Manuel test (Swagger) ✓ · PR review ✓